Diagnosing the Memory Throughput Bottleneck in the Era of Neural Rendering

Fangjun Zhou
Stanford University
fzhou48@stanford.edu

1 Introduction

Neural rendering methods such as NeRF [3] and 3D Gaussian Splatting (3DGS) [2] shows how traditional machine learning algorithms can be used to learn a scene representation from images and then render it from novel views. Early works like NeRF in this field have focused on improving the quality of the rendered images, but recent works have adopted ideas from compute graphics to improve the rendering speed. For example, 3DGS uses a point cloud representation and a rasterization-based rendering pipeline to achieve real-time rendering. InstantNGP [4] uses a multi-resolution hash encoding to speed up the training and rendering of NeRF. These methods have shown that it is possible to achieve high-quality training and rendering at interactive rates, but they also requires complicated data structures and render piplines to model the geometry, materials, and lighting transport in the scene. Implementing these neural rendering systems with traditional machine learning frameworks such as PyTorch and JAX is combersome, and shading languages or kernel libraries such as CUDA and OptiX do not have the automatic differentiation capabilities that are required for training these models. Therefore, the Slang shading language [1] is proposed to provide a unified programming model for neural rendering systems.

2 Slang Programming Model

Slang follows the same data-parallel programming model as traditional shading and kernel languages: the programmer writes an entry point function that is executed by many GPU threads in parallel, and each thread works on a different data element. For example, to implement a Slang kernel that adds two vectors, the programmer can define a kernel function as follows:

```
[shader("compute")]
[numthreads(256, 1, 1)]
void addMain(uint3 tid : SV_DispatchThreadID, StructuredBuffer<float> a,
↪  StructuredBuffer<float> b, RWStructuredBuffer<float> out, uint N)
{
    uint i = tid.x;
    if (i < N) out[i] = a[i] + b[i];
}
```

The host application can then dispatch a grid of threads to execute this kernel on the GPU, and each thread receives a built-in variable `tid` that indicates its unique thread ID. The programmer can use this thread ID to index into the input and output buffers and perform the desired computation.

2.1 Implementing MLPs in Slang

MLP is one of the most common building blocks in neural rendering systems, and it is composed of multiple linear layers and nonlinear activation functions. The throughput bottleneck of MLPs is often

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

the matrix multiplication operation in the linear layer. In Slang, a linear layer evaluation function can be implemented as follows:

```
public struct LinearLayer<int M, int N> : IDifferentiablePtrType
{
    typealias Differential = This;
    public RWStructuredBuffer<float> weight;
    public RWStructuredBuffer<float> bias;
}
[BackwardDerivative(backward)]
public float[M] forward(LinearLayer linearLayer, float[N] x)
{
    float[M] y;
    for (int i = 0; i < M; ++i)
    {
        float sum = linearLayer.bias[i];
        for (int j = 0; j < N; ++j)
            sum += linearLayer.weight[i * N + j] * x[j];
        y[i] = sum;
    }
    return y;
}
```

The `forward` function takes a linear layer and an input vector, and computes the output vector by performing a matrix multiplication and adding the bias. In Slang, the programmer can mark a function with the `[Differentiable]` attribute to indicate that it should be differentiated. Then the compiler can generate the reverse mode derivative propagation function via the `bwd_diff` operator. For example, the backward function for the linear layer can be implemented as follows:

```
public void backward<int M, int N>(DifferentialPtrPair<LinearLayer> layer,
↪   inout DifferentialPair<float[N]> x, float[M] grad)
{
    var dx = (float[N]).dzero();
    for (int i = 0; i < M; ++i)
    {
        layer.d.bias[i].add(grad[i]);
        for (int j = 0; j < N; ++j)
        {
            layer.d.weight[i * N + j].add(x.p[j] * grad[i]);
            dx[j] += layer.d.weight[i * N + j] * grad[i];
        }
    }
    x = diffPair(x.p, dx);
}
```

The `backward` function takes the gradient of the output and accumulates the gradients of the weights, bias, and input. During the backward pass, multiple threads may update the same weight or bias gradient, so it is important to use atomic operations to ensure correctness. Here, the `add` function is the atomic add operation that can be used to safely update the gradients in parallel.

3 Memory Throughput Bottleneck of Data-Parallel Programming Model

As mentioned previously, Slang uses a data-parallel programming model where each thread works on a different data element. This model is well-suited for many GPU workloads, but it can also lead to memory throughput bottlenecks when the threads need to access the same memory locations. In the case of evaluating MLPs, all the threads need to read the same weight and bias parameters, which can lead to memory bandwidth waste and reduced performance. This is especially problematic during the backward pass, where the threads need to write to the same weight and bias gradients with atomic operations, which can further reduce the performance.

Considering a linear layer with N_{in} input channels and N_{out} output channels being evaluated on T threads, the weight matrix $W \in \mathbb{R}^{N_{out} \times N_{in}}$ and bias vector $b \in \mathbb{R}^{N_{out}}$ are shared among all the threads. During the forward pass, each thread computes $y_i = Wx_i + b$ for its input x_i, where $i \in \{1, \ldots, T\}$ is the thread ID. This requires each thread to read the entire weight matrix and bias vector, which will result in $T \times (N_{out} \times N_{in} + N_{out})$ memory accesses in toal and can quickly saturate the memory bandwidth.

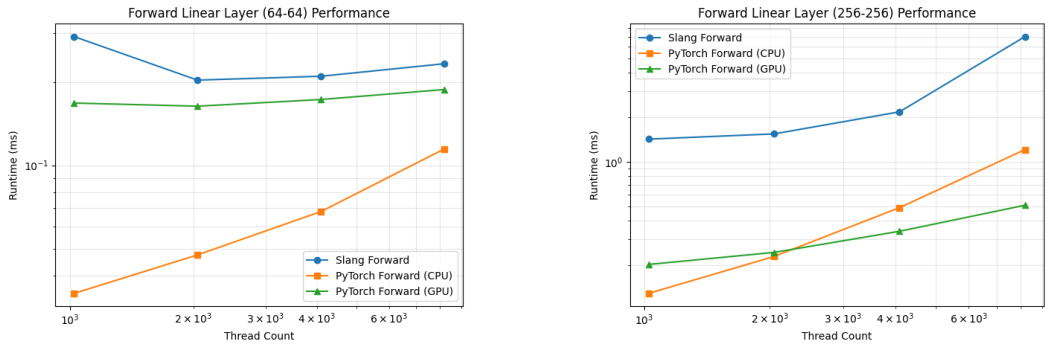
However, another way to formulate the linear layer evaluation is considering all the threads as a batch and performing a single matrix multiplication $Y = WX + b$, where $X \in \mathbb{R}^{N_{in} \times T}$ is the input matrix containing all the input vectors and $Y \in \mathbb{R}^{N_{out} \times T}$ is the output matrix. This formulation allows us to leverage optimized matrix multiplication libraries such as cuBLAS or CUTLASS, which can significantly reduce the memory bandwidth requirements by reusing the weight and bias parameters across multiple threads. If the weight and bias can be tiled and cached efficiently, the memory accesses for the weight and bias parameters can even be reduced close to $N_{out} \times N_{in} + N_{out}$, which can significantly improve the performance of the linear layer evaluation.

During the backward pass, considering the upstream gradient $\partial y_i \in \mathbb{R}^{N_{out}}$ for each thread, the gradients of the weights and bias can be computed as $\partial W = \sum_{i=1}^{T} \partial y_i x_i^T$ and $\partial b = \sum_{i=1}^{T} \partial y_i$, respectively. When implementing the gradient accumulation in a data-parallel manner, the programmer is essentially splitting a matrix matrix multiplication into multiple vector outer products across threads, which is inefficient in terms of memory bandwidth. Even worse, the threads need to use atomic operations to update the same weight and bias gradients, which can lead to contention and further reduce the performance. In contrast, by reformulating the backward pass as a batch matrix multiplication, we can leverage optimized linear algebra libraries to compute the gradients more efficiently and avoid the memory bandwidth bottleneck.

4 Experiments

To demonstrate the memory throughput bottleneck of the data-parallel programming model, I implement a simple linear layer both in Slang and PyTorch and compare the performance of the forward and backward passes on a M3 Pro MacBook with CPU for PyTorch and GPU both PyTorch and Slang.

The benchmark of the forward pass is shown in Figure 1.



(a) 64x64

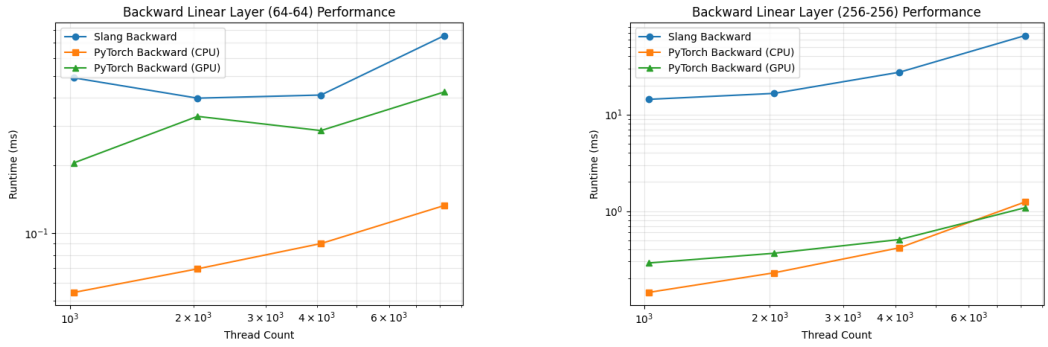
(b) 256x256

Figure 1: Forward pass performance across different linear layer sizes. All plots show runtime (ms) vs thread count on log-log scale, comparing Slang Forward, PyTorch Forward (CPU), and PyTorch Forward (GPU).

As shown in the figure, when the linear layer size is small (64x64), the performance of Slang and PyTorch on GPU are comparable, and both are significantly slower than PyTorch on CPU. This is likely because the overhead of launching GPU kernels and the memory bandwidth bottleneck outweigh the benefits of parallelism for small linear layers. However, when the linear layer size increases to 256x256, the performance of PyTorch on GPU starts to outperform PyTorch on CPU given a large enough thread count (batch size). For comparison, the performance of the naive

linear layer implementation in Slang does not improve much with increasing thread count, and it is significantly slower than PyTorch on GPU. As discussed previously, this is likely because the data-parallel implementation in Slang suffers from the memory throughput bottleneck, while PyTorch can leverage optimized matrix multiplication libraries to achieve better performance.

In Figure 2, I also benchmark the backward pass of the linear layer.



(a) 64x64

(b) 256x256

Figure 2: Backward pass performance across different linear layer sizes. All plots show runtime (ms) vs thread count on log-log scale, comparing Slang Backward, PyTorch Backward (CPU), and PyTorch Backward (GPU).

The backward pass shows even more pronounced performance differences. Specifically, the backward kernel is only 2.14x slower than the forward kernel in PyTorch on GPU with a 256×256 linear layer and a batch size of 8192. While in Slang, the backward kernel is 7.47x slower than the forward kernel with the same linear layer size and thread count. The memory throughput bottleneck is significantly worse during backpropagation because threads must use atomic operations to accumulate gradients into shared weight and bias buffers, leading to contention and serialization. PyTorch's optimized batch matrix multiplication approach avoids these issues by reformulating the gradient computation as efficient matrix operations.

References

[1] Sai Praveen Bangaru, Lifan Wu, Tzu-Mao Li, Jacob Munkberg, Gilbert Bernstein, Jonathan Ragan-Kelley, Frédo Durand, Aaron Lefohn, and Yong He. SLANG.D: Fast, Modular and Differentiable Shader Programming. *ACM Transactions on Graphics*, 42(6):1–28, December 2023.

[2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkuehler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4):1–14, August 2023.

[3] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, August 2020.

[4] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. *ACM Transactions on Graphics*, 41(4):1–15, July 2022.